

Black Box Linear Algebra

An Introduction to Wiedemann's Approach

William J. Turner

Department of Mathematics & Computer Science

Wabash College



Symbolic Computation



- Sometimes called Computer Algebra



Symbolic Computation



- Sometimes called Computer Algebra
- Symbols or exact arithmetic



Symbolic Computation



- Sometimes called Computer Algebra
- Symbols or exact arithmetic
- Not floating point numbers (numerical analysis)



Symbolic Computation



- Sometimes called Computer Algebra
- Symbols or exact arithmetic
- Not floating point numbers (numerical analysis)
- No round off errors



Symbolic Computation



- Sometimes called Computer Algebra
- Symbols or exact arithmetic
- Not floating point numbers (numerical analysis)
- No round off errors
- Algorithms may not be compatible



Symbolic Computation



- Sometimes called Computer Algebra
- Symbols or exact arithmetic
- Not floating point numbers (numerical analysis)
- No round off errors
- Algorithms may not be compatible
 - Numerical Analysis



Symbolic Computation



- Sometimes called Computer Algebra
- Symbols or exact arithmetic
- Not floating point numbers (numerical analysis)
- No round off errors
- Algorithms may not be compatible
 - Numerical Analysis
 - Find approximation quickly



Symbolic Computation



- Sometimes called Computer Algebra
- Symbols or exact arithmetic
- Not floating point numbers (numerical analysis)
- No round off errors
- Algorithms may not be compatible
 - Numerical Analysis
 - Find approximation quickly
 - May never find exact solution



Symbolic Computation



- Sometimes called Computer Algebra
- Symbols or exact arithmetic
- Not floating point numbers (numerical analysis)
- No round off errors
- Algorithms may not be compatible
 - Numerical Analysis
 - Find approximation quickly
 - May never find exact solution
 - Symbolic Computation



Symbolic Computation



- Sometimes called Computer Algebra
- Symbols or exact arithmetic
- Not floating point numbers (numerical analysis)
- No round off errors
- Algorithms may not be compatible
 - Numerical Analysis
 - Find approximation quickly
 - May never find exact solution
 - Symbolic Computation
 - Find exact solution quickly



Symbolic Computation



- Sometimes called Computer Algebra
- Symbols or exact arithmetic
- Not floating point numbers (numerical analysis)
- No round off errors
- Algorithms may not be compatible
 - Numerical Analysis
 - Find approximation quickly
 - May never find exact solution
 - Symbolic Computation
 - Find exact solution quickly
 - May never approximate



Probabilistic Algorithms



• Three types:



Probabilistic Algorithms



• Three types:

• *Monte Carlo*: Always fast, probably correct



Probabilistic Algorithms



• Three types:

• *Monte Carlo*: Always fast, probably correct

• *Las Vegas*: Probably fast, always correct



Probabilistic Algorithms



• Three types:

• *Monte Carlo*: Always fast, probably correct

• *Las Vegas*: Probably fast, always correct

• Certificate



Probabilistic Algorithms



• Three types:

• *Monte Carlo*: Always fast, probably correct

• *Las Vegas*: Probably fast, always correct

• Certificate

• *BPP*: Bounded Probabalistic Polynomial Time



Probabilistic Algorithms



• Three types:

• *Monte Carlo*: Always fast, probably correct

• *Las Vegas*: Probably fast, always correct

• Certificate

• *BPP*: Bounded Probabalistic Polynomial Time

• Probably fast, probably correct



Probabilistic Algorithms



• Three types:

• *Monte Carlo*: Always fast, probably correct

• *Las Vegas*: Probably fast, always correct

• Certificate

• *BPP*: Bounded Probabalistic Polynomial Time

• Probably fast, probably correct

• Atlantic City?



Probabilistic Algorithms



- Three types:

- Monte Carlo*: Always fast, probably correct

- Las Vegas*: Probably fast, always correct

- Certificate

- BPP*: Bounded Probabalistic Polynomial Time

- Probably fast, probably correct

- Atlantic City?

- Schwartz-Zippel Lemma



Probabilistic Algorithms



- Three types:

-  *Monte Carlo*: Always fast, probably correct

-  *Las Vegas*: Probably fast, always correct

-  Certificate

-  *BPP*: Bounded Probabalistic Polynomial Time

-  Probably fast, probably correct

-  Atlantic City?

-  Schwartz-Zippel Lemma

-  Probability randomly choose root of polynomial



Solving Linear Systems

$$\begin{bmatrix} 1 & 3 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 3 & 3 & -2 & 3 \\ 0 & -3 & 0 & -2 \end{bmatrix} x = \begin{bmatrix} 4 \\ 2 \\ 6 \\ -4 \end{bmatrix}$$

Gaussian Elimination

$$\begin{bmatrix} 1 & 3 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & 0 & -2 \end{bmatrix} x = \begin{bmatrix} 4 \\ 2 \\ 6 \\ 2 \end{bmatrix} \implies x = \begin{bmatrix} 1 \\ 2 \\ 0 \\ -1 \end{bmatrix}$$

- Row echelon form and back substitution

Gaussian Elimination

$$\begin{bmatrix} 1 & 3 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & 0 & -2 \end{bmatrix} x = \begin{bmatrix} 4 \\ 2 \\ 6 \\ 2 \end{bmatrix} \implies x = \begin{bmatrix} 1 \\ 2 \\ 0 \\ -1 \end{bmatrix}$$

- Row echelon form and back substitution
- Must know how matrix stored

Gaussian Elimination

$$\begin{bmatrix} 1 & 3 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & 0 & -2 \end{bmatrix} x = \begin{bmatrix} 4 \\ 2 \\ 6 \\ 2 \end{bmatrix} \implies x = \begin{bmatrix} 1 \\ 2 \\ 0 \\ -1 \end{bmatrix}$$

- Row echelon form and back substitution
- Must know how matrix stored
- Matrix changed in calculation

Gaussian Elimination

$$\begin{bmatrix} 1 & 3 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & 0 & -2 \end{bmatrix} x = \begin{bmatrix} 4 \\ 2 \\ 6 \\ 2 \end{bmatrix} \implies x = \begin{bmatrix} 1 \\ 2 \\ 0 \\ -1 \end{bmatrix}$$

- Row echelon form and back substitution
- Must know how matrix stored
- Matrix changed in calculation
- Sparse matrices may become dense

Black Box Matrix Model



- External view of matrix

Black Box Matrix Model



- External view of matrix
- Only matrix-vector products allowed

Black Box Matrix Model



- External view of matrix
- Only matrix-vector products allowed
- Independent of implementation

Black Box Matrix Model



- External view of matrix
- Only matrix-vector products allowed
- Independent of implementation
- Implementations efficient in time or space

Black Box Matrix Model



- External view of matrix
- Only matrix-vector products allowed
- Independent of implementation
- Implementations efficient in time or space
- Not unique to symbolic computation

Black Box Matrix Model



- External view of matrix
- Only matrix-vector products allowed
- Independent of implementation
- Implementations efficient in time or space
- Not unique to symbolic computation
- Can compute Krylov sequence $(A^i v)_{i \in \mathbb{N}}$

Examples

Matrix	storage	time
Arbitrary matrix	n^2	$O(n^2)$
Sparse matrix (η nonzero entries)	$O(\eta)$	$O(\eta)$
Hilbert matrix ($A^{[i,j]} = \frac{1}{i+j-1}$)	$O(1)$	$O(n)$
Toeplitz matrix	$O(n)$	$O(n \log(n) \log \log(n))$

Linearly Generated Sequences

Let $\mathbb{V}$ be a vector space over field $\mathbb{F}$.

A sequence

$$a = (a_i)_{i \in \mathbb{N}} \in \mathbb{V}^{\mathbb{N}}$$

is *linearly generated* if and only if there exist $m \in \mathbb{N}$ and

$$c_0, \dots, c_m \in \mathbb{F}, \quad c_m \neq 0$$

such that for all $i \geq 0$

$$\sum_{j=0}^m c_j a_{i+j} = 0$$

Linearly Generated Sequences

Let $\mathbb{V}$ be a vector space over field $\mathbb{F}$.

A sequence

$$a = (a_i)_{i \in \mathbb{N}} \in \mathbb{V}^{\mathbb{N}}$$

is *linearly generated* if and only if there exist $m \in \mathbb{N}$ and

$$c_0, \dots, c_m \in \mathbb{F}, \quad c_m \neq 0$$

such that for all $i \geq 0$

$$a_{i+m} = -\frac{1}{c_m} \left(\sum_{j=0}^{m-1} c_j a_{i+j} \right)$$

Generating Polynomials



- The polynomial $f(\lambda) = \sum_{j=0}^m c_j \lambda^j$ generates a



Generating Polynomials



• The polynomial $f(\lambda) = \sum_{j=0}^m c_j \lambda^j$ generates a

$$\bullet \quad f \bullet a = \left(\sum_{j=0}^m c_j a_{i+j} \right)_{i \in \mathbb{N}} = (0)_{i \in \mathbb{N}} = 0 \in \mathbb{V}^{\mathbb{N}}$$



Generating Polynomials



• The polynomial $f(\lambda) = \sum_{j=0}^m c_j \lambda^j$ generates a

$$\bullet \quad f \bullet a = \left(\sum_{j=0}^m c_j a_{i+j} \right)_{i \in \mathbb{N}} = (0)_{i \in \mathbb{N}} = 0 \in \mathbb{V}^{\mathbb{N}}$$

• $\mathbb{V}^{\mathbb{N}}$ is an $\mathbb{F}[\lambda]$ -module



Minimal Generating Polynomial



• If $f \bullet a = 0$ and $g \in \mathbb{F}[\lambda]$, then

$$(g f) \bullet a = g \bullet (f \bullet a) = g \bullet 0 = 0$$



Minimal Generating Polynomial



• If $f \bullet a = 0$ and $g \in \mathbb{F}[\lambda]$, then

$$(g f) \bullet a = g \bullet (f \bullet a) = g \bullet 0 = 0$$

• $\{f \in \mathbb{F}[\lambda] \mid f \bullet a = 0\}$ is an ideal



Minimal Generating Polynomial



• If $f \bullet a = 0$ and $g \in \mathbb{F}[\lambda]$, then

$$(g f) \bullet a = g \bullet (f \bullet a) = g \bullet 0 = 0$$

• $\{f \in \mathbb{F}[\lambda] \mid f \bullet a = 0\}$ is an ideal

• $\mathbb{F}[\lambda]$ is a principal ideal domain



Minimal Generating Polynomial



- If $f \bullet a = 0$ and $g \in \mathbb{F}[\lambda]$, then

$$(g f) \bullet a = g \bullet (f \bullet a) = g \bullet 0 = 0$$

- $\{f \in \mathbb{F}[\lambda] \mid f \bullet a = 0\}$ is an ideal
- $\mathbb{F}[\lambda]$ is a principal ideal domain
- There exists a unique monic generator of minimal degree, the *minimal generating polynomial* of sequence



Minimal Generating Polynomial



- If $f \bullet a = 0$ and $g \in \mathbb{F}[\lambda]$, then

$$(g f) \bullet a = g \bullet (f \bullet a) = g \bullet 0 = 0$$

- $\{f \in \mathbb{F}[\lambda] \mid f \bullet a = 0\}$ is an ideal
- $\mathbb{F}[\lambda]$ is a principal ideal domain
- There exists a unique monic generator of minimal degree, the *minimal generating polynomial* of sequence
- Minimal polynomial divides all generating polynomials



Example: Fibonacci Numbers



• $(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$



Example: Fibonacci Numbers



- $(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$
- Minimal polynomial is $f = \lambda^2 - \lambda - 1$



Example: Fibonacci Numbers



• $(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$

• Minimal polynomial is $f = \lambda^2 - \lambda - 1$

• $a_{i+2} = a_{i+1} + a_i$



Example: Fibonacci Numbers



- $(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$
- Minimal polynomial is $f = \lambda^2 - \lambda - 1$
 - $a_{i+2} = a_{i+1} + a_i$
- $(\lambda + 1)f = \lambda^3 - 2\lambda - 1$ also generates a



Example: Fibonacci Numbers



• $(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$

• Minimal polynomial is $f = \lambda^2 - \lambda - 1$

• $a_{i+2} = a_{i+1} + a_i$

• $(\lambda + 1)f = \lambda^3 - 2\lambda - 1$ also generates a

• $a_{i+3} = 2a_{i+1} + a_i$



Example: Fibonacci Numbers



- $(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$

- Minimal polynomial is $f = \lambda^2 - \lambda - 1$

- $a_{i+2} = a_{i+1} + a_i$

- $(\lambda + 1)f = \lambda^3 - 2\lambda - 1$ also generates a

- $a_{i+3} = 2a_{i+1} + a_i$

- $\lambda^k f = \lambda^{k+2} - \lambda^{k+1} - \lambda^k$ also generates a



Example: Fibonacci Numbers



- $(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$

- Minimal polynomial is $f = \lambda^2 - \lambda - 1$

- $a_{i+2} = a_{i+1} + a_i$

- $(\lambda + 1)f = \lambda^3 - 2\lambda - 1$ also generates a

- $a_{i+3} = 2a_{i+1} + a_i$

- $\lambda^k f = \lambda^{k+2} - \lambda^{k+1} - \lambda^k$ also generates a

- $a_{i+k+2} = a_{i+k+1} + a_{i+k}$



Example: Fibonacci Numbers



- $(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$

- Minimal polynomial is $f = \lambda^2 - \lambda - 1$

- $a_{i+2} = a_{i+1} + a_i$

- $(\lambda + 1)f = \lambda^3 - 2\lambda - 1$ also generates a

- $a_{i+3} = 2a_{i+1} + a_i$

- $\lambda^k f = \lambda^{k+2} - \lambda^{k+1} - \lambda^k$ also generates a

- $a_{i+k+2} = a_{i+k+1} + a_{i+k}$

- Skips first k elements of a



Matrix Power Sequence



$$\bullet \quad f \bullet (A^i)_{i \in \mathbb{N}} \iff f(A) = 0$$



Matrix Power Sequence



• $f \bullet (A^i)_{i \in \mathbb{N}} \iff f(A) = 0$

• $\det(\lambda I - A)$ generates $(A^i)_{i \in \mathbb{N}}$



Matrix Power Sequence



- $f \bullet (A^i)_{i \in \mathbb{N}} \iff f(A) = 0$
- $\det(\lambda I - A)$ generates $(A^i)_{i \in \mathbb{N}}$
 - Cayley-Hamilton Theorem



Matrix Power Sequence



- $f \bullet (A^i)_{i \in \mathbb{N}} \iff f(A) = 0$
- $\det(\lambda I - A)$ generates $(A^i)_{i \in \mathbb{N}}$
 - Cayley-Hamilton Theorem
- Let f^A be minimal polynomial of $(A^i)_{i \in \mathbb{N}}$



Matrix Power Sequence



- $f \bullet (A^i)_{i \in \mathbb{N}} \iff f(A) = 0$
- $\det(\lambda I - A)$ generates $(A^i)_{i \in \mathbb{N}}$
 - Cayley-Hamilton Theorem
- Let f^A be minimal polynomial of $(A^i)_{i \in \mathbb{N}}$
- $f^A \mid \det(\lambda I - A)$



Krylov Sequence



$$\bullet \quad f \bullet (A^i v)_{i \in \mathbb{N}} \iff f(A) v = 0$$



Krylov Sequence



• $f \bullet (A^i v)_{i \in \mathbb{N}} \iff f(A) v = 0$

• $f(A) = 0 \implies f(A) v = 0$



Krylov Sequence



• $f \bullet (A^i v)_{i \in \mathbb{N}} \iff f(A) v = 0$

• $f(A) = 0 \implies f(A) v = 0$

• $\{f \mid f \bullet (A^i)_{i \in \mathbb{N}} = 0\} \subset \{f \mid f \bullet (A^i v)_{i \in \mathbb{N}} = 0\}$



Krylov Sequence



- $f \bullet (A^i v)_{i \in \mathbb{N}} \iff f(A) v = 0$
- $f(A) = 0 \implies f(A) v = 0$
- $\{f \mid f \bullet (A^i)_{i \in \mathbb{N}} = 0\} \subset \{f \mid f \bullet (A^i v)_{i \in \mathbb{N}} = 0\}$
- Let $f^{A,v}$ be minimal polynomial of $(A^i v)_{i \in \mathbb{N}}$



Krylov Sequence



- $f \bullet (A^i v)_{i \in \mathbb{N}} \iff f(A) v = 0$
- $f(A) = 0 \implies f(A) v = 0$
- $\{f \mid f \bullet (A^i)_{i \in \mathbb{N}} = 0\} \subset \{f \mid f \bullet (A^i v)_{i \in \mathbb{N}} = 0\}$
- Let $f^{A,v}$ be minimal polynomial of $(A^i v)_{i \in \mathbb{N}}$
- $f^{A,v} \mid f^A \mid \det(\lambda I - A)$



Solving $Ax = b$

• Suppose $g = \sum_{j=0}^m c_j \lambda^j$, $g(0) = c_0 \neq 0$, and $g \bullet (A^i b)_{i \in \mathbb{N}}$

Solving $Ax = b$



- Suppose $g = \sum_{j=0}^m c_j \lambda^j$, $g(0) = c_0 \neq 0$, and $g \bullet (A^i b)_{i \in \mathbb{N}}$
- If g exists, then $f^{A,b}$ satisfies requirements



Solving $Ax = b$



- Suppose $g = \sum_{j=0}^m c_j \lambda^j$, $g(0) = c_0 \neq 0$, and $g \bullet (A^i b)_{i \in \mathbb{N}}$
- If g exists, then $f^{A,b}$ satisfies requirements
- $c_0 b + c_1 A b + \cdots + c_m A^m b = 0$



Solving $Ax = b$



- Suppose $g = \sum_{j=0}^m c_j \lambda^j$, $g(0) = c_0 \neq 0$, and $g \bullet (A^i b)_{i \in \mathbb{N}}$
- If g exists, then $f^{A,b}$ satisfies requirements
- $c_0 b + c_1 A b + \cdots + c_m A^m b = 0$
- $b = -\frac{1}{c_0} \sum_{j=1}^m (A^j b) = A \left(-\frac{1}{c_0} \sum_{j=1}^m (c_j A^{j-1} b) \right)$



Solving $Ax = b$



- Suppose $g = \sum_{j=0}^m c_j \lambda^j$, $g(0) = c_0 \neq 0$, and $g \bullet (A^i b)_{i \in \mathbb{N}}$
- If g exists, then $f^{A,b}$ satisfies requirements
- $c_0 b + c_1 A b + \cdots + c_m A^m b = 0$
- $b = -\frac{1}{c_0} \sum_{j=1}^m (A^j b) = A \left(-\frac{1}{c_0} \sum_{j=1}^m (c_j A^{j-1} b) \right)$
- $x = -\frac{1}{c_0} \sum_{j=1}^m (c_j A^{j-1} b)$ is a solution



Nonsingular A



- $\det(A) \neq 0$



Nonsingular A



- $\det(A) \neq 0$
- $\det(\lambda I - A)|_{\lambda=0} \neq 0$



Nonsingular A



- $\det(A) \neq 0$
- $\det(\lambda I - A)|_{\lambda=0} \neq 0$
- $f^{A,b}(0) \neq 0$



Nonsingular A



- $\det(A) \neq 0$
- $\det(\lambda I - A)|_{\lambda=0} \neq 0$
- $f^{A,b}(0) \neq 0$
- $x = -\frac{1}{c_0} \sum_{j=1}^m (c_j A^{j-1} b)$ is the unique solution



Bilinear Projection Sequence



$$\bullet \quad f \bullet (u^\top A^i v)_{i \in \mathbb{N}} \iff u^\top f(A) v = 0$$



Bilinear Projection Sequence



• $f \bullet (u^\top A^i v)_{i \in \mathbb{N}} \iff u^\top f(A) v = 0$

• $f(A) v = 0 \implies u^\top f(A) v = 0$



Bilinear Projection Sequence



• $f \bullet (u^\top A^i v)_{i \in \mathbb{N}} \iff u^\top f(A) v = 0$

• $f(A) v = 0 \implies u^\top f(A) v = 0$

• $\{f \mid f \bullet (A^i v)_{i \in \mathbb{N}} = 0\} \subset \{f \mid f \bullet (u^\top A^i v)_{i \in \mathbb{N}} = 0\}$



Bilinear Projection Sequence



- $f \bullet (u^\top A^i v)_{i \in \mathbb{N}} \iff u^\top f(A) v = 0$
- $f(A) v = 0 \implies u^\top f(A) v = 0$
- $\{f \mid f \bullet (A^i v)_{i \in \mathbb{N}} = 0\} \subset \{f \mid f \bullet (u^\top A^i v)_{i \in \mathbb{N}} = 0\}$
- Let $f_u^{A,v}$ be minimal polynomial of $(u^\top A^i v)_{i \in \mathbb{N}}$



Bilinear Projection Sequence



- $f \bullet (u^\top A^i v)_{i \in \mathbb{N}} \iff u^\top f(A) v = 0$
- $f(A) v = 0 \implies u^\top f(A) v = 0$
- $\{f \mid f \bullet (A^i v)_{i \in \mathbb{N}} = 0\} \subset \{f \mid f \bullet (u^\top A^i v)_{i \in \mathbb{N}} = 0\}$
- Let $f_u^{A,v}$ be minimal polynomial of $(u^\top A^i v)_{i \in \mathbb{N}}$
- $f_u^{A,v} \mid f^{A,v} \mid f^A \mid \det(\lambda I - A)$



Bilinear Projection Sequence



- $f \bullet (u^\top A^i v)_{i \in \mathbb{N}} \iff u^\top f(A) v = 0$
- $f(A) v = 0 \implies u^\top f(A) v = 0$
- $\{f \mid f \bullet (A^i v)_{i \in \mathbb{N}} = 0\} \subset \{f \mid f \bullet (u^\top A^i v)_{i \in \mathbb{N}} = 0\}$
- Let $f_u^{A,v}$ be minimal polynomial of $(u^\top A^i v)_{i \in \mathbb{N}}$
- $f_u^{A,v} \mid f^{A,v} \mid f^A \mid \det(\lambda I - A)$
- If u chosen randomly from finite $S^n \subset \mathbb{F}^n$, then $f_u^{A,v} = f^{A,v}$ with probability at least

$$1 - \frac{\deg(f^{A,v})}{|S|}$$



Bilinear Projection Sequence



- $f \bullet (u^\top A^i v)_{i \in \mathbb{N}} \iff u^\top f(A) v = 0$
- $f(A) v = 0 \implies u^\top f(A) v = 0$
- $\{f \mid f \bullet (A^i v)_{i \in \mathbb{N}} = 0\} \subset \{f \mid f \bullet (u^\top A^i v)_{i \in \mathbb{N}} = 0\}$
- Let $f_u^{A,v}$ be minimal polynomial of $(u^\top A^i v)_{i \in \mathbb{N}}$
- $f_u^{A,v} \mid f^{A,v} \mid f^A \mid \det(\lambda I - A)$
- If u chosen randomly from finite $S^n \subset \mathbb{F}^n$, then $f_u^{A,v} = f^{A,v}$ with probability at least

$$1 - \frac{\deg(f^{A,v})}{|S|}$$

- Certificate: $f_u^{A,v} \bullet (A^i v)_{i \in \mathbb{N}} = 0$



Computing Minimal Polynomial



- Must know $\deg(f) < M$



Computing Minimal Polynomial



- Must know $\deg(f) < M$
- Extended Euclidean Algorithm



Computing Minimal Polynomial



- Must know $\deg(f) < M$
- Extended Euclidean Algorithm
 - $s_j f_{-1} + t_j f_0 = f_j$



Computing Minimal Polynomial



- Must know $\deg(f) < M$
- Extended Euclidean Algorithm
 - $s_j f_{-1} + t_j f_0 = f_j$
 - Inputs are $f_{-1}(\lambda) = \sum_{i=0}^{2M-1} (a_i \lambda^i)$ and $f_0(\lambda) = \lambda^{2M}$



Computing Minimal Polynomial



- Must know $\deg(f) < M$
- Extended Euclidean Algorithm
 - $s_j f_{-1} + t_j f_0 = f_j$
 - Inputs are $f_{-1}(\lambda) = \sum_{i=0}^{2M-1} (a_i \lambda^i)$ and $f_0(\lambda) = \lambda^{2M}$
 - Stop when $\deg(f_j) < M < \deg(f_{j-1})$



Computing Minimal Polynomial



- Must know $\deg(f) < M$
- Extended Euclidean Algorithm
 - $s_j f_{-1} + t_j f_0 = f_j$
 - Inputs are $f_{-1}(\lambda) = \sum_{i=0}^{2M-1} (a_i \lambda^i)$ and $f_0(\lambda) = \lambda^{2M}$
 - Stop when $\deg(f_j) < M < \deg(f_{j-1})$
 - Minimal polynomial is reversal of $s_j(\lambda)$



Computing Minimal Polynomial



- Must know $\deg(f) < M$
- Extended Euclidean Algorithm
 - $s_j f_{-1} + t_j f_0 = f_j$
 - Inputs are $f_{-1}(\lambda) = \sum_{i=0}^{2M-1} (a_i \lambda^i)$ and $f_0(\lambda) = \lambda^{2M}$
 - Stop when $\deg(f_j) < M < \deg(f_{j-1})$
 - Minimal polynomial is reversal of $s_j(\lambda)$
- Berlekamp-Massey Algorithm



Computing Minimal Polynomial



- Must know $\deg(f) < M$
- Extended Euclidean Algorithm
 - $s_j f_{-1} + t_j f_0 = f_j$
 - Inputs are $f_{-1}(\lambda) = \sum_{i=0}^{2M-1} (a_i \lambda^i)$ and $f_0(\lambda) = \lambda^{2M}$
 - Stop when $\deg(f_j) < M < \deg(f_{j-1})$
 - Minimal polynomial is reversal of $s_j(\lambda)$
- Berlekamp-Massey Algorithm
 - From coding theory



Computing Minimal Polynomial



- Must know $\deg(f) < M$
- Extended Euclidean Algorithm
 - $s_j f_{-1} + t_j f_0 = f_j$
 - Inputs are $f_{-1}(\lambda) = \sum_{i=0}^{2M-1} (a_i \lambda^i)$ and $f_0(\lambda) = \lambda^{2M}$
 - Stop when $\deg(f_j) < M < \deg(f_{j-1})$
 - Minimal polynomial is reversal of $s_j(\lambda)$
- Berlekamp-Massey Algorithm
 - From coding theory
 - Interpolates elements of scalar sequence



Computing Minimal Polynomial



- Must know $\deg(f) < M$
- Extended Euclidean Algorithm
 - $s_j f_{-1} + t_j f_0 = f_j$
 - Inputs are $f_{-1}(\lambda) = \sum_{i=0}^{2M-1} (a_i \lambda^i)$ and $f_0(\lambda) = \lambda^{2M}$
 - Stop when $\deg(f_j) < M < \deg(f_{j-1})$
 - Minimal polynomial is reversal of $s_j(\lambda)$
- Berlekamp-Massey Algorithm
 - From coding theory
 - Interpolates elements of scalar sequence
 - Related to Extended Euclidean Algorithm



Berlekamp-Massey Algorithm

Input: Scalar sequence $(a_i)_{i \in \mathbb{N}} \in \mathbb{F}^{\mathbb{N}}$ with generator g with
 $\deg(g) \leq m$

Output: Minimal polynomial f of sequence

- 1: $f \leftarrow 1$ {Initial guess}
- 2: **for** $r = 0$ to $2m - 1$ **do** $\{f$ generates $a_0, \dots, a_{r-1}\}$
- 3: $f = c_0 + c_1 \lambda + \dots + c_d \lambda^d$
- 4: $\Delta \leftarrow c_0 a_{r-d} + c_1 a_{r-d+1} + \dots + c_d a_r$
- 5: **if** $\Delta \neq 0$ **then** $\{f$ does not generate $a_0, \dots, a_r\}$
- 6: update f to generate $a_0, \dots, a_r$
- 7: **end if**
- 8: **end for**

Example: Fibonacci Numbers

$$(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$$

<u>r</u>	<u>Δ</u>	<u>recursion relation</u>
0	0	$a_j = 0$

Example: Fibonacci Numbers

$$(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$$

<u>r</u>	<u>Δ</u>	<u>recursion relation</u>
0	0	$a_j = 0$
1	1	$a_{j+2} = a_j$

Example: Fibonacci Numbers

$$(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$$

<u>r</u>	<u>Δ</u>	<u>recursion relation</u>
0	0	$a_j = 0$
1	1	$a_{j+2} = a_j$
2	1	$a_{j+2} = a_{j+1} + a_j$

Example: Fibonacci Numbers

$$(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$$

<u>r</u>	<u>Δ</u>	<u>recursion relation</u>
0	0	$a_j = 0$
1	1	$a_{j+2} = a_j$
2	1	$a_{j+2} = a_{j+1} + a_j$
3	0	$a_{j+2} = a_{j+1} + a_j$

Example: Fibonacci Numbers

$$(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$$

<u>r</u>	<u>Δ</u>	<u>recursion relation</u>
0	0	$a_j = 0$
1	1	$a_{j+2} = a_j$
2	1	$a_{j+2} = a_{j+1} + a_j$
3	0	$a_{j+2} = a_{j+1} + a_j$
4	0	$a_{j+2} = a_{j+1} + a_j$

Example: Fibonacci Numbers

$$(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$$

<u>r</u>	<u>Δ</u>	<u>recursion relation</u>
0	0	$a_j = 0$
1	1	$a_{j+2} = a_j$
2	1	$a_{j+2} = a_{j+1} + a_j$
3	0	$a_{j+2} = a_{j+1} + a_j$
4	0	$a_{j+2} = a_{j+1} + a_j$
5	0	$a_{j+2} = a_{j+1} + a_j$

Example: Fibonacci Numbers

$$(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$$

<u>r</u>	<u>Δ</u>	<u>recursion relation</u>
0	0	$a_j = 0$
1	1	$a_{j+2} = a_j$
2	1	$a_{j+2} = a_{j+1} + a_j$
3	0	$a_{j+2} = a_{j+1} + a_j$
4	0	$a_{j+2} = a_{j+1} + a_j$
5	0	$a_{j+2} = a_{j+1} + a_j$
6	0	$a_{j+2} = a_{j+1} + a_j$

Example: Fibonacci Numbers

$$(a_i)_{i \in \mathbb{N}} = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots)$$

<u>r</u>	<u>Δ</u>	<u>recursion relation</u>
0	0	$a_j = 0$
1	1	$a_{j+2} = a_j$
2	1	$a_{j+2} = a_{j+1} + a_j$
3	0	$a_{j+2} = a_{j+1} + a_j$
4	0	$a_{j+2} = a_{j+1} + a_j$
5	0	$a_{j+2} = a_{j+1} + a_j$
6	0	$a_{j+2} = a_{j+1} + a_j$
7	0	$a_{j+2} = a_{j+1} + a_j$

Wiedemann's Algorithm



• $f_u^{A,v} \mid f^{A,v} \mid f^A \mid \det(\lambda I - A)$



Wiedemann's Algorithm



- $f_u^{A,v} \mid f^{A,v} \mid f^A \mid \det(\lambda I - A)$
- $\deg(f_u^{A,v}) \leq \deg(\det(\lambda I - A)) = n$



Wiedemann's Algorithm



- $f_u^{A,v} \mid f^{A,v} \mid f^A \mid \det(\lambda I - A)$
- $\deg(f_u^{A,v}) \leq \deg(\det(\lambda I - A)) = n$
- Only need $(u^\top A^i b)_{i=0}^{2n-1}$



Wiedemann's Algorithm



- $f_u^{A,v} \mid f^{A,v} \mid f^A \mid \det(\lambda I - A)$
- $\deg(f_u^{A,v}) \leq \deg(\det(\lambda I - A)) = n$
- Only need $(u^\top A^i b)_{i=0}^{2n-1}$

Require: nonsingular $A \in \mathbb{F}^{n \times n}$ and $b \in \mathbb{F}^n$

Ensure: $x \in \mathbb{F}^n$ such that $Ax = b$

1: $u \leftarrow$ random vector in S^n where $S \subset \mathbb{F}$

2: use Berlekamp-Massey to compute

$$f^{A,b} = c_0 + c_1\lambda + \cdots + c_m\lambda^m \quad \{\text{Store only } A^i b\}$$

3: $x \leftarrow -\frac{1}{c_0}(c_1b + c_2Ab + \cdots c_m A^{m-1}b)$



Back to original problem




$$\begin{bmatrix} 1 & 3 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 3 & 3 & -2 & 3 \\ 0 & -3 & 0 & -2 \end{bmatrix} x = \begin{bmatrix} 4 \\ 2 \\ 6 \\ -4 \end{bmatrix}$$



Back to original problem



•
$$\begin{bmatrix} 1 & 3 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 3 & 3 & -2 & 3 \\ 0 & -3 & 0 & -2 \end{bmatrix} x = \begin{bmatrix} 4 \\ 2 \\ 6 \\ -4 \end{bmatrix}$$

• $n = 4$



Back to original problem



•
$$\begin{bmatrix} 1 & 3 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 3 & 3 & -2 & 3 \\ 0 & -3 & 0 & -2 \end{bmatrix} x = \begin{bmatrix} 4 \\ 2 \\ 6 \\ -4 \end{bmatrix}$$

• $n = 4$

• $u = \begin{bmatrix} 2 \\ 1 \\ 1 \\ 2 \end{bmatrix}$



Back to original problem

$$\bullet (u^T A^i b)_{i=0}^{2n-1} = (8, -4, 20, -28, 68, -124, 260, -508)$$

Back to original problem



- $(u^\top A^i b)_{i=0}^{2n-1} = (8, -4, 20, -28, 68, -124, 260, -508)$
- $f^{A,b} = f_u^{A,b} = \lambda^2 + \lambda - 2$



Back to original problem



- $(u^T A^i b)_{i=0}^{2n-1} = (8, -4, 20, -28, 68, -124, 260, -508)$

- $f^{A,b} = f_u^{A,b} = \lambda^2 + \lambda - 2$

- $$x = \frac{1}{2} (Ab + b) = \frac{1}{2} \left(\begin{bmatrix} -2 \\ 2 \\ -6 \\ 2 \end{bmatrix} + \begin{bmatrix} 4 \\ 2 \\ 6 \\ -4 \end{bmatrix} \right) = \begin{bmatrix} 1 \\ 2 \\ 0 \\ -1 \end{bmatrix}$$



Singular A

Kaltofen and Saunders (1991):

• If

Singular A

Kaltofen and Saunders (1991):

• If

• $r = \text{rank}(A) < n$ known

Singular A

Kaltofen and Saunders (1991):

• If

• $r = \text{rank}(A) < n$ known

• $r \times r$ leading principal minor nonzero

Singular A

Kaltofen and Saunders (1991):

• If

• $r = \text{rank}(A) < n$ known

• $r \times r$ leading principal minor nonzero

• Randomly choose v

Singular A

Kaltofen and Saunders (1991):

• If

- $r = \text{rank}(A) < n$ known
- $r \times r$ leading principal minor nonzero
- Randomly choose v
- Solve $r \times r$ nonsingular system $A_r y'_{b,v} = A'(b + A v)$

Singular A

Kaltofen and Saunders (1991):

• If

- $r = \text{rank}(A) < n$ known
- $r \times r$ leading principal minor nonzero
- Randomly choose v
- Solve $r \times r$ nonsingular system $A_r y'_{b,v} = A'(b + A v)$
 - A_r is leading $r \times r$ submatrix of A

Singular A

Kaltofen and Saunders (1991):

• If

- $r = \text{rank}(A) < n$ known
- $r \times r$ leading principal minor nonzero
- Randomly choose v
- Solve $r \times r$ nonsingular system $A_r y'_{b,v} = A'(b + A v)$
 - A_r is leading $r \times r$ submatrix of A
 - A' is first r rows of A

Singular A

Kaltofen and Saunders (1991):

• If

• $r = \text{rank}(A) < n$ known

• $r \times r$ leading principal minor nonzero

• Randomly choose v

• Solve $r \times r$ nonsingular system $A_r y'_{b,v} = A'(b + A v)$

• A_r is leading $r \times r$ submatrix of A

• A' is first r rows of A

• Then $y_{b,v} - v = \begin{bmatrix} y'_{b,v} \\ 0 \end{bmatrix} - v$ uniformly samples solution manifold

Precondition Matrix



- Want $r \times r$ leading principal minor of $\tilde{A}$ nonzero



Precondition Matrix



- Want $r \times r$ leading principal minor of $\tilde{A}$ nonzero
- $\tilde{A} = BA$ or $\tilde{A} = AB$



Precondition Matrix



- Want $r \times r$ leading principal minor of $\tilde{A}$ nonzero
- $\tilde{A} = BA$ or $\tilde{A} = AB$
- Need efficient matrix-vector product



Precondition Matrix



- Want $r \times r$ leading principal minor of $\tilde{A}$ nonzero
- $\tilde{A} = BA$ or $\tilde{A} = AB$
- Need efficient matrix-vector product
- New linear system $\tilde{A}\tilde{x} = \tilde{b}$



Generic Rank Profile



🕒 *Wiedemann (1986):* $\tilde{A} = A P$, P parameterized and can realize any permutation



Generic Rank Profile



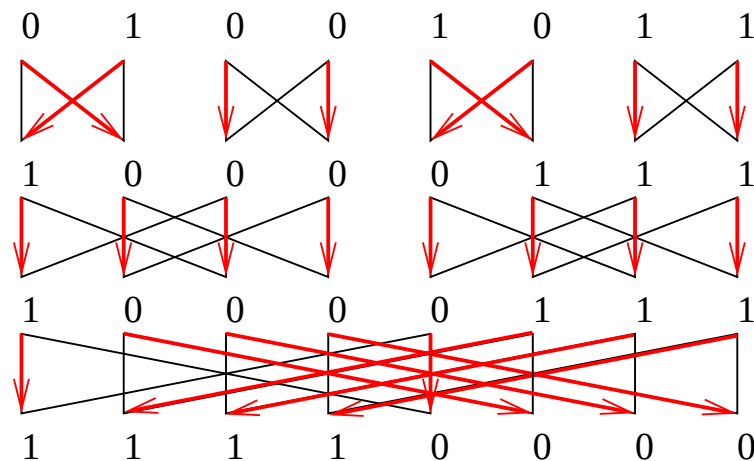
- *Wiedemann (1986)*: $\tilde{A} = A P$, P parameterized and can realize any permutation
- *Kaltofen and Saunders (1991)*: $\tilde{A} = T_1 A T_2$, T_1 unit upper triangular Toeplitz and T_2 unit lower triangular Toeplitz



Generic Rank Profile



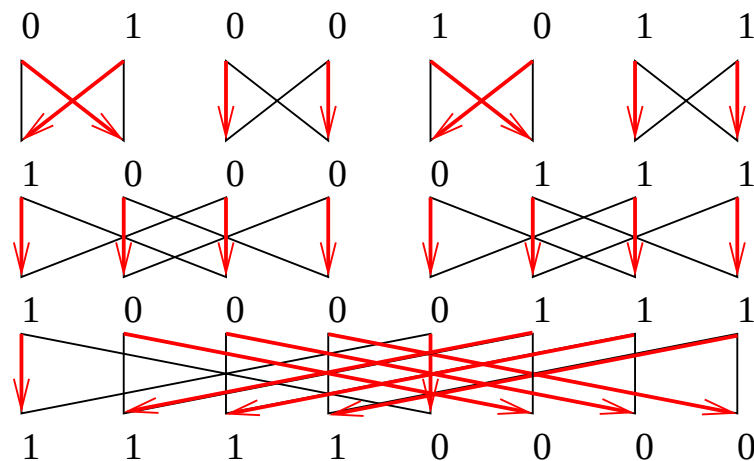
- Wiedemann (1986): $\tilde{A} = A P$, P parameterized and can realize any permutation
- Kaltofen and Saunders (1991): $\tilde{A} = T_1 A T_2$, T_1 unit upper triangular Toeplitz and T_2 unit lower triangular Toeplitz
- Chen et al. (2002); Turner (2002): $\tilde{A} = B_1 A B_2$, B_1, B_2 based on butterfly networks



Generic Rank Profile



- Wiedemann (1986): $\tilde{A} = A P$, P parameterized and can realize any permutation
- Kaltofen and Saunders (1991): $\tilde{A} = T_1 A T_2$, T_1 unit upper triangular Toeplitz and T_2 unit lower triangular Toeplitz
- Chen et al. (2002); Turner (2002): $\tilde{A} = B_1 A B_2$, B_1 , B_2 based on butterfly networks



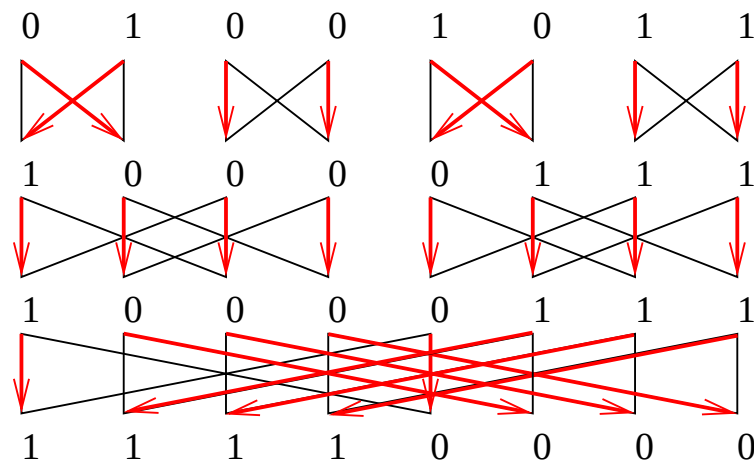
- Generic exchange matrix mixes inputs



Generic Rank Profile



- Wiedemann (1986): $\tilde{A} = A P$, P parameterized and can realize any permutation
- Kaltofen and Saunders (1991): $\tilde{A} = T_1 A T_2$, T_1 unit upper triangular Toeplitz and T_2 unit lower triangular Toeplitz
- Chen et al. (2002); Turner (2002): $\tilde{A} = B_1 A B_2$, B_1 , B_2 based on butterfly networks



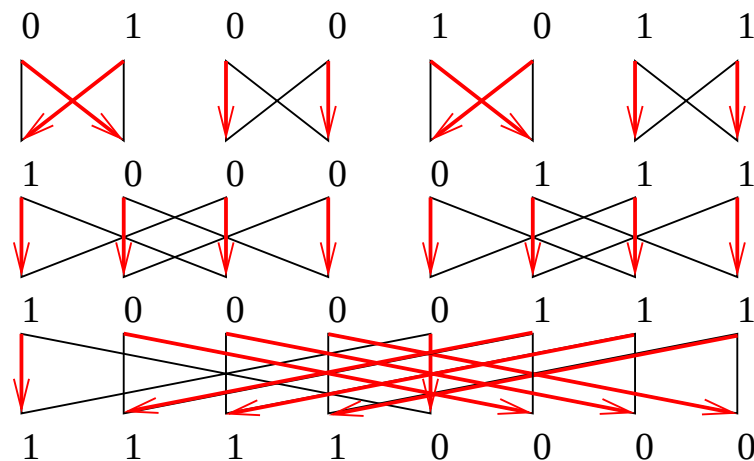
- Generic exchange matrix mixes inputs
- Turner (2002): Generalize butterfly networks to radix- β switches



Generic Rank Profile



- Wiedemann (1986): $\tilde{A} = A P$, P parameterized and can realize any permutation
- Kaltofen and Saunders (1991): $\tilde{A} = T_1 A T_2$, T_1 unit upper triangular Toeplitz and T_2 unit lower triangular Toeplitz
- Chen et al. (2002); Turner (2002): $\tilde{A} = B_1 A B_2$, B_1 , B_2 based on butterfly networks



- Generic exchange matrix mixes inputs
- Turner (2002): Generalize butterfly networks to radix- β switches
- Patton and Turner (2004): Toeplitz (or Hankel) matrix switches



Matrix Rank



Kaltofen and Saunders (1991):

• If



Matrix Rank



Kaltofen and Saunders (1991):

• If

• $r = \text{rank}(A) < n$ (unknown)



Matrix Rank



Kaltofen and Saunders (1991):

• If

• $r = \text{rank}(A) < n$ (unknown)

• Leading principal minors nonzero up to A_r



Matrix Rank

Kaltofen and Saunders (1991):

• If

- $r = \text{rank}(A) < n$ (unknown)
- Leading principal minors nonzero up to A_r
- $D = \text{diag}(d_1, \dots, d_n)$

Matrix Rank

Kaltofen and Saunders (1991):

• If

• $r = \text{rank}(A) < n$ (unknown)

• Leading principal minors nonzero up to A_r

• $D = \text{diag}(d_1, \dots, d_n)$

• Then $r = \deg(f^{AD}) - 1$ with probability at least $1 - \frac{n(n-1)}{2|S|}$

Matrix Minimal Polynomial



• If v chosen randomly, then $f^{A,v} = f^A$ with probability at least

$$1 - \frac{\deg(f^A)}{|S|} \geq 1 - \frac{n}{|S|}$$



Matrix Minimal Polynomial



- If v chosen randomly, then $f^{A,v} = f^A$ with probability at least

$$1 - \frac{\deg(f^A)}{|S|} \geq 1 - \frac{n}{|S|}$$

- If u and v chosen randomly, then $f_u^{A,v} = f^A$ with probability at least

$$1 - \frac{\deg(2 f^A)}{|S|} \geq 1 - \frac{2n}{|S|}$$



Rank Preconditioners



- Generic rank profile preconditioner and diagonal matrix



Rank Preconditioners



- Generic rank profile preconditioner and diagonal matrix
- *Chen et al. (2002)*: $\tilde{A} = T_3 A T_4 T_5$
 T_3 and T_4 unit lower triangular Toeplitz, T_5 upper triangular Toeplitz



Rank Preconditioners



- Generic rank profile preconditioner and diagonal matrix
- *Chen et al. (2002)*: $\tilde{A} = T_3 A T_4 T_5$
 T_3 and T_4 unit lower triangular Toeplitz, T_5 upper triangular Toeplitz
- *Turner (2002, 2003)*: Relax slightly generic rank profile



Rank Preconditioners



- Generic rank profile preconditioner and diagonal matrix
- *Chen et al. (2002)*: $\tilde{A} = T_3 A T_4 T_5$
 T_3 and T_4 unit lower triangular Toeplitz, T_5 upper triangular Toeplitz
- *Turner (2002, 2003)*: Relax slightly generic rank profile

- *Patton and Turner (2004)*: $\tilde{A} = A T$ $\begin{bmatrix} d_1 & & \\ & \ddots & \\ & & d_n \end{bmatrix}$

T Toeplitz (or Hankel)



Rank Preconditioners



- Generic rank profile preconditioner and diagonal matrix
- *Chen et al. (2002)*: $\tilde{A} = T_3 A T_4 T_5$
 T_3 and T_4 unit lower triangular Toeplitz, T_5 upper triangular Toeplitz
- *Turner (2002, 2003)*: Relax slightly generic rank profile

- *Patton and Turner (2004)*: $\tilde{A} = A T$ $\begin{bmatrix} d_1 & & \\ & \ddots & \\ & & d_n \end{bmatrix}$

T Toeplitz (or Hankel)

- *Patton and Turner (2004)*: $\tilde{A} = A T$



Matrix Determinant



• If $f^A = \det(\lambda I - A)$



Matrix Determinant



- If $f^A = \det(\lambda I - A)$
- Then $\det(A) = (-1)^n f^A(0)$



Determinant Preconditioners



- Generic rank profile preconditioner and diagonal matrix



Determinant Preconditioners



- Generic rank profile preconditioner and diagonal matrix
- *Kaltofen and Pan (1992):* $\tilde{A} = T_1 A T_2$
 T_1 unit upper triangular Toeplitz and T_2 lower triangular Toeplitz



Determinant Preconditioners



- Generic rank profile preconditioner and diagonal matrix
- *Kaltofen and Pan (1992)*: $\tilde{A} = T_1 A T_2$
 T_1 unit upper triangular Toeplitz and T_2 lower triangular Toeplitz

- *Chen et al. (2002)*: $\tilde{A} = A \begin{bmatrix} d_1 & & \\ & \ddots & \\ & & d_n \end{bmatrix}$



Determinant Preconditioners



- Generic rank profile preconditioner and diagonal matrix

- *Kaltofen and Pan (1992)*: $\tilde{A} = T_1 A T_2$

T_1 unit upper triangular Toeplitz and T_2 lower triangular Toeplitz

- *Chen et al. (2002)*: $\tilde{A} = A \begin{bmatrix} d_1 & & \\ & \ddots & \\ & & d_n \end{bmatrix}$

- *Turner (2002, 2003)*: $\tilde{A} = A \begin{bmatrix} 1 & a_1 & & \\ & \ddots & \ddots & \\ & & 1 & a_{n-1} \\ & & & 1 \end{bmatrix}$



References

- E. R. BERLEKAMP (1968). *Algebraic Coding Theory*. McGraw-Hill, New York.
- L. CHEN, W. EBERLY, E. KALTOFEN, B. D. SAUNDERS, W. J. TURNER, and G. VILLARD (2002). Efficient Matrix Preconditioners for Black Box Linear Algebra. *Linear Algebra and its Applications*, **343-344**: 119–146. Special issue on *Infinite Systems of Linear Equations Finitely Specified*.
- J. L. DORNSTETTER (1987). On the Equivalence Between Berlekamp's and Euclid's Algorithms. *IEEE Transactions on Information Theory*, **IT-33**(3): 428–431.
- J. VON ZUR GATHEN and J. GERHARD (1999). *Modern Computer Algebra*. Cambridge University Press, Cambridge.
- E. KALTOFEN (1992). Efficient Solution of Sparse Linear Systems. Lecture notes, Computer Science Department, Rensselaer Polytechnic Institute, Troy, N.Y.
- E. KALTOFEN and V. PAN (1991). Processor Efficient Parallel Solution of Linear Systems over an Abstract Field. In *Proceedings of SPAA '91 3rd Annual ACM Symposium on Parallel Algorithms and Architectures*, 180–191. ACM Press, New York, New York.
- E. KALTOFEN and V. PAN (1992). Processor Efficient Parallel Solution of Linear Systems II: the Positive Characteristic and Singular Cases. In *Proceedings of the 33rd Annual Symposium on Foundations of Computer Science*, 714–723. IEEE Computer Society Press, Los Alamitos, California.
- E. KALTOFEN and B. D. SAUNDERS (1991). On Wiedemann's Method of Solving Sparse Linear Systems. In H. F. MATTSON, T. MORA, and T. R. N. RAO (eds.), *AAECC-9: Proceedings of the 1991 Applied Algebra, Algebraic Algorithms and Error-Correcting Codes, International Conference*, vol. 539 of *Lecture Notes in Computer Science*, 29–38. Springer Verlag, Heidelberg, Germany.
- J. L. MASSEY (1969). Shift-Register Synthesis and BCH Decoding. *IEEE Transactions on Information Theory*, **IT-15**(1): 122–127.
- A. E. PATTON and W. J. TURNER (2004). Toeplitz and Hankel Matrix Preconditioners for Black Box Linear Algebra. To be submitted to International Symposium on Symbolic and Algebraic Computation 2004.
- J. T. SCHWARTZ (1980). Fast Probabilistic Algorithms for Verification of Polynomial Identities. *J. ACM*, **27**: 701–717.
- W. J. TURNER (2002). *Black Box Linear Algebra with the LinBox Library*. Ph.D. thesis, North Carolina State University, Raleigh, North Carolina.
- W. J. TURNER (2003). Determinantal Divisors and Matrix Preconditioners. Submitted to Journal of Symbolic Computation.
- D. H. WIEDEMANN (1986). Solving Sparse Linear Equations Over Finite Fields. *IEEE Transactions on Information Theory*, **IT-32**(1): 54–62.
- R. ZIPPEL (1979). Probabilistic Algorithms for Sparse Polynomials. In E. W. NG (ed.), *Symbolic and Algebraic Computation, EUROSAM '79, An International Symposium on Symbolic and Algebraic Computation, Marseille, France, June 1979, Proceedings*, vol. 72 of *Lecture Notes in Computer Science*, 216–226. Springer Verlag, Heidelberg, Germany.
- R. ZIPPEL (1990). Interpolating Polynomials from Their Values. *Journal of Symbolic Computation*, **9**(3): 375–403.